

执行与报告

本篇讲案例是怎么被跑起来的，以及执行完之后怎么读报告。理解执行机制有助于判断一次失败到底是被测系统的问题，还是案例本身的问题。

执行器工作原理

架构

ATP 的执行是**平台和执行器分离**的：

...

ATP Web (写案例) MySQL Executor (读案例、驱动浏览器) Selenium Grid

...

Web 端只负责写和查，不碰浏览器。执行器是独立的服务，轮询数据库拿待执行的任务，把案例翻译成 Selenium 操作。

这个分离带来一个实际影响：**案例保存后立即生效，不需要发布或同步**。但反过来，正在执行中的案例如果被改了，这一轮跑的是执行器已经读进内存的旧版本。

执行器资源

执行器数量有限，并发跑不满时任务会排队。可以在「执行任务」页看到当前队列。

单个执行器同一时刻只跑一条案例。所以**同一条案例不会并发执行**，不用担心自己和自己抢数据。但**不同案例会并发**，如果两条案例操作同一份测试数据，就可能互相干扰 —— 这是偶发失败的一个常见来源，排查时值得怀疑。

浏览器选择

`browser` 字段决定用哪个浏览器，默认 `CHROME`。

Grid 里 Chrome 的节点最多，Firefox 和 Edge 各只有少量节点。所以**指定 Firefox / Edge 的案例排队时间会明显更长**。除非确实要验证浏览器兼容性，否则保持默认。

失败分类

平台把失败归为四类，**分类不同排查方向完全不同**，看报告时先看这个：

ELEMENT_NOT_FOUND

在等待超时内没有找到符合定位器的元素。

这类失败**大概率是案例的问题，不是被测系统的 bug**。排查顺序：

1. 看失败截图 —— 页面长什么样？是不是根本没到该到的页面
2. 是不是在 iframe 里而没有 `SWITCH\_FRAME`
3. 是不是点开了新窗口而没有 `SWITCH\_WINDOW`
4. 定位器是不是用了会失效的写法（绝对路径、动态 id、索引下标）

TIMEOUT

元素找到了，但一直不满足等待条件；或者整条案例超过了 `timeout\_sec`。

要区分两种：

- **步骤超时** —— 元素在，但一直不可见 / 不可点。常见于按钮一直 disabled（前置操作没做对）、或者遮罩层一直没消失（被测系统慢或卡住）
- **案例超时** —— 步骤都正常，只是总时间超了。看看是不是步骤太多、或者 `timeout\_sec` 设得太紧

ASSERT_FAILED

元素找到了、也可见，但内容不符合预期。

****这类失败最有价值 —— 它才是最可能真正发现 bug 的。****

前两类多半是案例自身的问题，`ASSERT\_FAILED` 则说明系统的实际行为和预期不一致。

看到这类失败，先别急着改案例，认真判断是预期错了还是系统错了。

SCRIPT_ERROR

执行器内部错误，比如浏览器崩溃、Grid 节点掉线、案例数据不合法。

这类和被测系统无关，重跑一次通常就好。如果反复出现，报给平台组。

报告怎么读

排查顺序

按这个顺序看能少走弯路：

1. ****失败分类**** —— 先定方向，四类的排查路径完全不同
2. ****失败步骤的截图**** —— 八成问题一眼能看出来。ATP 在每个步骤失败时自动截图，截的是失败那一刻的完整页面
3. ****前一个成功步骤的截图**** —— 这一步经常被忽略，但它能告诉你「失败前页面是什么状态」，对判

断流程是否走对非常有用

4. **完整日志** —— 前面几步定位不到再看

截图的局限

截图是**失败那一刻**的，不是失败原因发生时的。

典型例子：第 4 步点击没生效，第 7 步断言失败并截图。截图上看到的是「页面停在原地」，但真正的问题在第 4 步。所以看到断言失败时，要往前追溯操作步骤是否真的生效了。

这也是为什么建议给关键操作后面加断言 —— 让失败尽早暴露在它真正发生的地方。

失败重试

任务级重试

执行任务可以配置失败重试，默认关闭。开启后失败的案例会自动重跑，最多 2 次。

重试只应该用来对抗环境抖动，不应该用来掩盖不稳定的案例。

判断标准：如果一条案例需要靠重试才能进入通过率统计，它就是一条坏案例，应该修，而不是靠重试凑数。

用重试率发现坏案例

报告里有个「首次通过率」和「最终通过率」的区分。两者差距大的案例就是**不稳定案例**，值得优先治理。

平台每周会生成一份不稳定案例榜单，排在前面的通常有共同特征：用了 `SLEEP`、用了 `VISIBLE` 而非 `CLICKABLE` 去点击、或者定位器依赖了动态属性。

常见的执行期问题

案例在批量任务里不跑

九成是 `status` 还是 `DRAFT`。批量执行只捡 `ACTIVE` 的案例。

本地能过，CI 上失败

通常是时序问题。本地机器空闲、网络快，元素很快就绪；CI 上并发高、机器慢，同样的等待时间就不够了。

这类问题的正确修法**不是调大超时**，而是检查等待策略是不是写对了 —— 尤其是 `CLICK` 有没有用 `CLICKABLE`。详见《等待策略》。

偶发失败，重跑就好

排查方向按可能性排序：

1. ****等待策略写错**** —— 最常见
2. ****案例间数据冲突**** —— 两条案例并发操作同一份数据
3. ****测试数据被污染**** —— 上一轮执行改了数据没恢复
4. ****被测系统确实不稳定**** —— 这种情况值得报 bug，而不是改案例迁就它

第 4 种经常被忽略。自动化测试发现的偶发问题，往往对应着真实用户偶尔会遇到的故障。****不要为了让案例变绿而掩盖它。****